
Tecnologie del Linguaggio Naturale

Domande e Risposte – Parte 1

Università degli Studi di Torino (UniTO)
Corso di Laurea in Informatica
A.A. 2025/26

Alessandro Olivero

Documento generato a scopo di studio

0. Contents

1	Come funziona il Transition Based Parser (MALT)?	3
1.1	Configurazione iniziale e finale	3
1.2	Le tre operazioni (Arc-Standard)	3
1.3	Oracle e modello statistico	3
2	Spiegare l'uso delle HMM nel PoS Tagging	3
2.1	Componenti del modello	3
2.2	Obiettivo	4
2.3	Algoritmo di Viterbi	4
2.4	Stima dei parametri	4
3	Che differenza c'è tra MEMM e HMM?	4
3.1	MEMM – Maximum Entropy Markov Model	4
3.2	Label Bias Problem	5
4	Cosa è una Probabilistic CFG (PCFG)?	5
4.1	Probabilità di un albero sintattico	5
4.2	Utilizzo	5
4.3	Limitazioni	5
5	Cosa è il task della Aggregation?	5
5.1	Problema	6
5.2	Esempio	6
5.3	Tecniche	6
6	I 5 algoritmi per il parsing delle dipendenze sintattiche	6
7	Il formalismo Lambda-FoL per la Semantica Computazionale	7
7.1	Motivazione	7
7.2	Lambda astrazione e applicazione	7
7.3	Esempio	7
7.4	Integrazione con la sintassi	7
8	La rappresentazione Neo-Davidsoniana	7
8.1	Semantica Davidsoniana (base)	8
8.2	Neo-Davidsonismo	8
8.3	Vantaggi	8
9	Le due ipotesi fondamentali della linguistica computazionale	8
10	La Frame-Based Semantics nei Dialogue Systems	8
10.1	Struttura di un frame	9
10.2	Funzionamento nel dialogo	9
10.3	Esempio	9
11	Esempio di ambiguità puramente semantica	9
11.1	Esempi classici	9

12 Ambiguità sintattica: PP attachment e coordinazione	10
12.1 PP Attachment (Attacco del Sintagma Preposizionale)	10
12.2 Ambiguità della Coordinazione	10
13 Come funziona l'algoritmo CKY?	10
13.1 Pre-requisito: conversione in CNF	10
13.2 Algoritmo (programmazione dinamica)	11
13.3 Esempio rapido	11
14 Complessità dell'algoritmo CKY	11
14.1 Analisi	11
14.2 Rispetto alle alternative	11
15 Dipendenze vs. Costituenti	11
15.1 Convertibilità	12
16 La misura PARSEVAL	12
16.1 Definizione	12
16.2 Labeled vs. Unlabeled	12
16.3 Limitazioni	12
17 Cosa è un Treebank?	12
17.1 Caratteristiche	13
17.2 Esempi principali	13
17.3 Utilizzo	13
18 HMM: modello generativo o discriminativo?	13
18.1 Spiegazione	13
18.2 Confronto	13
19 Anatomia di un Parser	13
19.1 Componenti principali	14
19.2 Flusso di esecuzione	14
20 I 6 fenomeni notevoli del dialogo tra umani	14
21 Cosa è il PoS Tagging?	15
21.1 Classi principali	15
21.2 Esempio (Universal POS tagset)	15
21.3 Sfide	15
21.4 Approcci	15
22 Cosa è il NER Tagging?	15
22.1 Classi tipiche (CoNLL-2003)	15
22.2 Schema di annotazione (IOB2)	16
22.3 Approcci	16
22.4 Applicazioni	16

1. Come funziona il Transition Based Parser (MALT)?

Il **MALT Parser** (Maltparser) è un parser per la sintassi a dipendenze basato su un meccanismo di *transizioni*. Il processo di parsing è modellato come una sequenza di azioni su una configurazione composta da:

- **Stack** (σ): contiene i token già processati parzialmente.
- **Buffer** (β): contiene i token ancora da processare (la frase in input).
- **Insieme di archi** (A): contiene le relazioni di dipendenza già costruite.

Configurazione iniziale e finale

- **Inizio**: stack con il solo token ROOT, buffer con l'intera frase.
- **Fine**: buffer vuoto, tutte le dipendenze costruite in A .

Le tre operazioni (Arc-Standard)

- **SHIFT**: sposta il primo elemento del buffer in cima allo stack.
- **LEFT-ARC**(r): crea un arco $(\text{top}(\sigma), \text{sec}(\sigma), r)$, ovvero il top dello stack diventa la testa del secondo elemento; il secondo viene rimosso dallo stack.
- **RIGHT-ARC**(r): crea un arco $(\text{sec}(\sigma), \text{top}(\sigma), r)$; il top viene rimosso dallo stack.

Oracle e modello statistico

In fase di training, un *oracle* (basato su un gold treebank) guida le transizioni corrette. A runtime, un **classificatore** (SVM, perceptron o rete neurale) predice la prossima transizione basandosi su feature estratte dalla configurazione corrente (es. le POS dei token in cima allo stack e in testa al buffer).

La complessità è **lineare** $O(n)$ rispetto alla lunghezza n della frase, poiché ogni token viene shiftato e rimosso dallo stack al più una volta.

2. Spiegare l'uso delle HMM nel PoS Tagging

Un **Hidden Markov Model** (HMM) modella il PoS Tagging come un problema di *decodifica*: dato il testo osservato $w_1, \dots, w_n$, si vuole trovare la sequenza di tag $t_1, \dots, t_n$ più probabile.

Componenti del modello

- **Stati nascosti**: i tag POS (t_i).
- **Osservazioni**: le parole (w_i).
- **Probabilità di transizione**: $P(t_i | t_{i-1})$ – la probabilità che un tag segua il precedente.

- **Probabilità di emissione:** $P(w_i | t_i)$ – la probabilità che un tag “emetta” una parola.
- **Probabilità iniziale:** $P(t_1)$.

Obiettivo

$$\hat{t}_{1:n} = \arg \max_{t_{1:n}} P(t_1) \cdot \prod_{i=2}^n P(t_i | t_{i-1}) \cdot \prod_{i=1}^n P(w_i | t_i)$$

Algoritmo di Viterbi

La decodifica efficiente è realizzata con l'algoritmo di **Viterbi** (programmazione dinamica), che evita di enumerare esponenzialmente tutte le sequenze possibili, operando in tempo $O(n \cdot |T|^2)$ dove $|T|$ è la dimensione del tagset.

Stima dei parametri

I parametri vengono stimati da un corpus annotato tramite *Maximum Likelihood Estimation*:

$$P(t_i | t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}, \quad P(w_i | t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

Lo **smoothing** (es. add-one, back-off) è necessario per gestire gli zeri.

3. Che differenza c'è tra MEMM e HMM?

Aspetto	HMM	MEMM
Tipo di modello	Generativo	Discriminativo
Distribuzione modellata	$P(w, t)$	$P(t w)$
Feature	Solo parola corrente	Feature arbitrarie sul contesto
Training	Conteggi + smoothing	Ottimizzazione (es. L-BFGS)
Label bias	No	Sì (problema noto)
Complessità	Bassa	Maggiore

MEMM – Maximum Entropy Markov Model

Il MEMM modella direttamente la distribuzione condizionale:

$$P(t_i | t_{i-1}, w_{1:n}) = \frac{1}{Z} \exp \left(\sum_k \lambda_k f_k(t_i, t_{i-1}, w_{1:n}, i) \right)$$

dove f_k sono feature arbitrarie (suffisso della parola, capitalizzazione, parola precedente, ecc.) e λ_k sono i pesi appresi.

Label Bias Problem

Il principale svantaggio del MEMM è il **label bias problem**: le probabilità di transizione sono normalizzate localmente per ogni stato, favorendo gli stati con pochi successori. Le **CRF** (Conditional Random Fields) risolvono questo problema con una normalizzazione globale.

4. Cosa è una Probabilistic CFG (PCFG)?

Una **Probabilistic Context-Free Grammar** (PCFG) estende una CFG associando una probabilità a ogni regola di produzione.

Una PCFG è una tupla $G = (N, \Sigma, R, S, P)$ dove:

- N : insieme dei non-terminali
- Σ : insieme dei terminali
- R : regole di produzione $A \rightarrow \alpha$
- S : simbolo iniziale
- P : funzione che assegna $P(A \rightarrow \alpha)$ a ogni regola, con $\sum_{\alpha} P(A \rightarrow \alpha) = 1$

Probabilità di un albero sintattico

La probabilità di un albero T è il prodotto delle probabilità delle regole usate:

$$P(T) = \prod_{(A \rightarrow \alpha) \in T} P(A \rightarrow \alpha)$$

Utilizzo

Le PCFG vengono usate per il **disambiguation sintattico**: tra tutti gli alberi compatibili con la grammatica si sceglie quello con probabilità massima. I parametri si stimano da un treebank con MLE: $P(A \rightarrow \alpha) = C(A \rightarrow \alpha)/C(A)$.

Limitazioni

Le PCFG assumono indipendenza dal contesto (la probabilità di una regola non dipende dal contesto in cui appare il non-terminale), il che porta a stime poco accurate. Estensioni come le **Lexicalized PCFG** (Collins, Charniak) mitigano questo problema.

5. Cosa è il task della Aggregation?

L'**Aggregation** è uno dei task fondamentali della **Natural Language Generation** (NLG). Si occupa di decidere *come raggruppare e combinare* le informazioni da comunicare in unità testuali coerenti.

Problema

Dato un insieme di fatti o proposizioni da esprimere, l'aggregation stabilisce:

- Quante frasi produrre.
- Quali proposizioni unire nella stessa frase (es. coordinazione, subordinazione).
- Come evitare la ripetizione (es. tramite ellissi o pronominalizzazione).

Esempio

Input: {“Mario è studente”, “Mario studia informatica”, “Mario abita a Torino”}

Senza aggregation: “Mario è studente. Mario studia informatica. Mario abita a Torino.”

Con aggregation: “Mario è uno studente di informatica che abita a Torino.”

Tecniche

Le principali strategie di aggregation includono:

- **Set aggregation:** raggruppamento di proposizioni con stesso soggetto o predicato.
- **Conjunction:** unione con connettivi coordinanti.
- **Embedding:** inserimento di una proposizione in un'altra come relativa o participiale.

6. I 5 algoritmi per il parsing delle dipendenze sintattiche

1. Transition-Based Parsing (MALT)

Approccio greedy basato su transizioni (SHIFT, LEFT-ARC, RIGHT-ARC). Complessità lineare $O(n)$. Dipende da un classificatore per scegliere la prossima azione. Vedi sezione 1 per dettagli.

2. Graph-Based Parsing (MST – Eisner/McDonald)

Il parsing è formulato come ricerca del *Maximum Spanning Tree* su un grafo completo pesato dei token. L'algoritmo di **Chu-Liu/Edmonds** (per grafi diretti) o **Eisner** (per alberi proiettivi) trova l'albero ottimale globale. Complessità $O(n^2)$ o $O(n^3)$.

3. CYK-Based Dependency Parsing

Adattamento del CYK per dipendenze. Richiede una fase di conversione tra rappresentazioni (costituenti $\leftrightarrow$ dipendenze). Meno comune nella pratica moderna.

4. Easy-First Parsing

Variante del transition-based che processa prima le decisioni più “facili” (alta confidenza del classificatore) anziché seguire un ordine sinistro-destro fisso. Riduce la propagazione degli errori.

5. Neural Dependency Parsing (Deep Biaffine)

Approccio moderno di **Dozat & Manning (2017)**: utilizza una rete BiLSTM + meccanismo *biaffine* per calcolare un punteggio per ogni possibile arco head $\rightarrow$

dep e per ogni etichetta. Il decoding avviene con MST. Stato dell'arte con feature contestualizzate (BERT, ecc.).

7. Il formalismo Lambda-FoL per la Semantica Computazionale

Il **Lambda Calcolo applicato alla Logica del Primo Ordine** (*Lambda-FoL*) è un formalismo per associare rappresentazioni semantiche composizionali alle strutture sintattiche.

Motivazione

Il principio di **composizionalità di Frege** afferma che il significato di un'espressione è funzione del significato delle sue parti. Il lambda calcolo permette di rappresentare significati come funzioni componibili.

Lambda astrazione e applicazione

- **Lambda astrazione:** $\lambda x. \varphi(x)$ rappresenta una funzione che, dato x , restituisce $\varphi(x)$.
- **Beta-riduzione:** $(\lambda x. \varphi(x))(a) \Rightarrow \varphi(a)$.

Esempio

Considera la frase “Every student reads a book”:

$$\begin{aligned} \text{“reads”} &\rightsquigarrow \lambda y. \lambda x. \text{reads}(x, y) \\ \text{“a book”} &\rightsquigarrow \lambda P. \exists y [\text{book}(y) \wedge P(y)] \\ \text{“every student”} &\rightsquigarrow \lambda Q. \forall x [\text{student}(x) \Rightarrow Q(x)] \end{aligned}$$

Componendo via applicazione funzionale si ottiene la formula FoL:

$$\forall x [\text{student}(x) \Rightarrow \exists y [\text{book}(y) \wedge \text{reads}(x, y)]]$$

Integrazione con la sintassi

Il formalismo si integra con grammatiche categoriali (CCG) o con regole compositive su alberi sintattici, assegnando a ciascun lessema un'entrata semantica in lambda-FoL che viene composta bottom-up.

8. La rappresentazione Neo-Davidsoniana

La rappresentazione **Neo-Davidsoniana** è un'estensione della semantica Davidsoniana classica per gestire in modo uniforme gli argomenti verbali e i modificatori.

Semantica Davidsoniana (base)

Davidson (1967) propose di introdurre una **variabile evento** e nelle formule logiche. Ad esempio:

$$\text{“Mario corre velocemente”} \rightsquigarrow \exists e[\text{correre}(e) \wedge \text{agente}(e, \text{Mario}) \wedge \text{veloce}(e)]$$

Neo-Davidsonismo

L’approccio Neo-Davidsoniano (Parsons 1990) tratta **tutti gli argomenti tematici** come predicati sull’evento, separando completamente la struttura argomentale:

$$\text{“Mario mangia una mela”} \rightsquigarrow \exists e[\text{mangiare}(e) \wedge \theta_{ag}(e, \text{Mario}) \wedge \theta_{th}(e, \text{mela})]$$

dove θ_{ag} e θ_{th} sono i ruoli tematici *agente* e *tema*.

Vantaggi

- Trattamento uniforme di argomenti obbligatori e modificatori opzionali.
- Facilita l’inferenza (es. da “Mario mangia una mela velocemente” si può inferire “Mario mangia una mela”).
- Si integra bene con risorse come VerbNet e PropBank.

9. Le due ipotesi fondamentali della linguistica computazionale

I. Ipotesi di regolarità (o *ipotesi linguistica*):

Il linguaggio naturale è strutturato e governato da regolarità sistematiche – fonologiche, morfologiche, sintattiche, semantiche – che possono essere descritte formalmente e sfruttate computazionalmente.

II. Ipotesi di corpora (o *ipotesi statistica/empirica*):

I dati linguistici reali (corpora) contengono informazione sufficiente per apprendere, stimare e valutare modelli del linguaggio. Il comportamento linguistico è in larga misura apprendibile dall’osservazione di grandi quantità di testo.

Queste due ipotesi giustificano rispettivamente l’approccio **simbolico/rule-based** (grammatiche, lexicon) e l’approccio **statistico/empirico** (n-gram, HMM, reti neurali) che convivono e si integrano nella linguistica computazionale moderna.

10. La Frame-Based Semantics nei Dialogue Systems

La **Frame-Based Semantics** nei *Dialogue Systems* (DS) è un approccio alla gestione della comprensione del linguaggio naturale basato sul concetto di **frame**: strutture dati predefinite che rappresentano domini e intenzioni dell’utente.

Struttura di un frame

Un frame è composto da:

- **Intent (intenzione)**: l'azione che l'utente vuole compiere (es. `prenota_volo`).
- **Slot**: i campi parametrici da riempire (es. `destinazione`, `data`, `classe`).
- **Valori**: i valori effettivi estratti dall'enunciato (es. `Roma`, `15 giugno`, `economica`).

Funzionamento nel dialogo

1. L'**NLU** (Natural Language Understanding) identifica l'intent e riempie i valori degli slot.
2. Il **Dialogue Manager** tiene traccia degli slot mancanti (*slot filling*) e gestisce i turni.
3. L'**NLG** genera le domande per ottenere i valori mancanti (es. "Qual è la data di partenza?").
4. Quando tutti gli slot obbligatori sono riempiti, viene eseguita l'azione.

Esempio

Utente: "Voglio volare a Milano domani sera"

Frame `prenota_volo`: `destinazione=Milano`, `data=domani`, `fascia_oraria=sera`, `partenza=??` (mancante → sistema chiede).

11. Esempio di ambiguità puramente semantica

L'ambiguità **puramente semantica** si manifesta quando una frase ha una struttura sintattica unica ma consente più interpretazioni semantiche.

Esempi classici

1. **Polisemia lessicale**:
 "Ho visto **la banca**."
 → istituto finanziario *oppure* riva di un fiume. La struttura sintattica (SN oggetto diretto) è identica in entrambi i casi.
2. **Ambiguità di scope quantificazionale**:
 "Ogni studente ha letto un libro."
 → *Lettura 1*: $\forall x[stud(x) \Rightarrow \exists y[libro(y) \wedge legge(x, y)]]$ (ogni studente ha letto un libro qualsiasi, possibilmente diverso)
 → *Lettura 2*: $\exists y[libro(y) \wedge \forall x[stud(x) \Rightarrow legge(x, y)]]$ (c'è un libro specifico che tutti hanno letto)
3. **Ambiguità del pronome (coreference)**:
 "Mario ha detto a Luca che **lui** aveva ragione."
 → "lui" = Mario *oppure* Luca.

12. Ambiguità sintattica: PP attachment e coordinazione

PP Attachment (Attacco del Sintagma Preposizionale)

Il **PP attachment** è uno dei fenomeni di ambiguità sintattica più studiati. Un sintagma preposizionale può essere associato (“attaccato”) a diversi costituenti nella frase, cambiando il significato.

Esempio canonico: “I saw the man with the telescope.”

- **Attacco al VP:** ho usato il telescopio per vedere l’uomo.
- **Attacco all’NP:** ho visto l’uomo che aveva il telescopio.

In italiano: “Ho mangiato la torta con la forchetta” (strumento) vs. “Ho mangiato la torta con la ciliegia” (parte della torta).

Il PP attachment è un problema statistico: la risoluzione dipende dal contesto semantico e lessicale, non solo dalla struttura sintattica. I modelli moderni usano informazioni di *selezional restrictions* e dati di co-occorrenza.

Ambiguità della Coordinazione

La coordinazione con “e” (o “and”) può legarsi a costituenti di livello diverso, producendo interpretazioni distinte.

Esempio: “Ragazze e ragazzi intelligenti”

- Lettura 1: $[ragazze]$ e $[ragazzi intelligenti]$ $\rightarrow$ solo i ragazzi sono intelligenti.
- Lettura 2: $[ragazze intelligenti]$ e $[ragazzi intelligenti]$ $\rightarrow$ entrambi sono intelligenti.

Esempio strutturale: “Ho mangiato pasta e bevuto vino e acqua”

- $[pasta]$ e $[bevuto vino e acqua]$ (coordinazione di VP e NP)
- Varie strutture ad albero possibili.

13. Come funziona l’algoritmo CKY?

Il **Cocke-Kasami-Younger (CKY)** è un algoritmo di parsing a *chart* per grammatiche in **Forma Normale di Chomsky (CNF)**: ogni regola è della forma $A \rightarrow BC$ oppure $A \rightarrow w$.

Pre-requisito: conversione in CNF

Qualsiasi CFG può essere convertita in CNF. I passi principali sono:

- Eliminare le ε -produzioni.
- Eliminare le produzioni unitarie ($A \rightarrow B$).
- Binarizzare le produzioni con più di 2 simboli.

Algoritmo (programmazione dinamica)

Si costruisce una **tabella triangolare** $T[i][j]$ che indica i non-terminali che derivano la sottostringa $w_{i+1} \dots w_j$.

Inizializzazione: per ogni i , $T[i][i+1] = \{A \mid A \rightarrow w_{i+1} \in R\}$.

Ricorsione: per $l = 2, \dots, n$ (lunghezza); $i = 0, \dots, n-l$; $j = i+l$:

$$T[i][j] = \bigcup_{k=i+1}^{j-1} \{A \mid A \rightarrow BC \in R, B \in T[i][k], C \in T[k][j]\}$$

Risultato: la frase è riconosciuta se $S \in T[0][n]$.

Esempio rapido

Per la frase “she eats fish” con regole appropriate, si riempie la tabella bottom-up partendo dagli span di lunghezza 1, poi 2, poi l'intero span.

14. Complessità dell'algoritmo CKY

Complessità temporale: $O(n^3 \cdot |G|)$, dove n è la lunghezza dell'input e $|G|$ è la dimensione della grammatica (numero di regole).

Analisi

I tre loop annidati sono:

- l – lunghezza dello span: n valori.
- i – posizione di inizio: $O(n)$ valori.
- k – punto di split: $O(n)$ valori.

Per ogni tripla (i, j, k) si controllano tutte le regole $A \rightarrow BC$: $O(|G|)$.

Rispetto alle alternative

Il CKY è **polinomiale** ed è ottimale per CFG generali. Algoritmi come Earley hanno complessità $O(n^3)$ nel caso peggiore ma $O(n^2)$ per grammatiche non ambigue. In pratica, le grammatiche linguisticamente realistiche rendono il CKY pesante su frasi lunghe, spingendo verso parser approssimativi o neurali.

15. Dipendenze vs. Costituenti

Le due principali rappresentazioni della struttura sintattica sono le **grammatiche a costituenti** (phrase structure) e le **grammatiche a dipendenze**.

Aspetto	Costituenti	Dipendenze
Struttura	Albero con nodi interni (NP, VP...)	Albero con archi testa→dipendente
Nodi	Terminali + non-terminali	Solo terminali (le parole)
Head	Non esplicita (richiede lessicalizzazione)	Esplicita per ogni arco
Ordine	Dipende fortemente dall'ordine delle parole	Più robusto a variazioni d'ordine
Risorse	Penn Treebank (PTB), ATIS	Universal Dependencies (UD)
Parser tipici	CKY, Earley, chart parser	MALT, MST, biaffine

Convertibilità

Le due rappresentazioni sono parzialmente intercambiabili: algoritmi come *head-percolation* permettono di estrarre dipendenze da un albero a costituenti, e viceversa. Le **Universal Dependencies** (UD) sono oggi il formato standard per le dipendenze multilinguali.

16. La misura PARSEVAL

PARSEVAL è la metrica standard per valutare i parser a costituenti (phrase structure parser). È stata introdotta da Black et al. (1991).

Definizione

Dato un albero prodotto dal parser (*proposed*) e l'albero di riferimento (*gold*), si contano i **costituenti** corretti, dove un costituente è identificato da una tripla (non-terminale, inizio, fine).

$$\text{Precision} = \frac{|\text{proposti corretti}|}{|\text{proposti}|} \quad \text{Recall} = \frac{|\text{proposti corretti}|}{|\text{gold}|} \quad F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

Labeled vs. Unlabeled

- **Labeled:** la tripla include l'etichetta del non-terminale (es. NP, VP).
- **Unlabeled:** conta solo la posizione dello span, ignorando l'etichetta.

Limitazioni

PARSEVAL non penalizza allo stesso modo tutti gli errori (errori sugli span interni sono contati meno) ed è sensibile alle scelte di annotazione del treebank. Per la sintassi a dipendenze si usano invece *UAS* (Unlabeled Attachment Score) e *LAS* (Labeled Attachment Score).

17. Cosa è un Treebank?

Un **treebank** è un corpus di testo annotato con struttura sintattica (alberi), creato manualmente da linguisti esperti o semi-automaticamente tramite un parser seguito

da revisione umana.

Caratteristiche

- Ogni frase è associata a un albero sintattico (a costituenti o a dipendenze).
- Le annotazioni includono spesso anche informazioni morfologiche e PoS tag.
- La creazione è costosa (tipicamente 1–2 ore per frase nelle prime versioni).

Esempi principali

Treebank	Lingua	Tipo
Penn Treebank (PTB)	Inglese	Costituenti
Universal Dependencies (UD)	100+ lingue	Dipendenze
Turin University Treebank	Italiano	Costituenti
Prague Dependency Treebank	Ceco	Dipendenze

Utilizzo

I treebank sono fondamentali per: (1) training di parser supervisionati, (2) stima dei parametri di PCFG, (3) valutazione (gold standard per PARSEVAL, UAS, LAS).

18. HMM: modello generativo o discriminativo?

HMM è un modello *generativo*.

Spiegazione

Un modello generativo modella la **distribuzione congiunta** $P(X, Y)$ dove X sono le osservazioni e Y le etichette. Dall'HMM si può *generare* (campionare) una sequenza di tag e una sequenza di parole secondo il processo:

$$P(w_{1:n}, t_{1:n}) = P(t_1) \prod_{i=2}^n P(t_i | t_{i-1}) \prod_{i=1}^n P(w_i | t_i)$$

Confronto

- **Generativo** (HMM, Naive Bayes): modella $P(X, Y)$; classificazione via Bayes.
- **Discriminativo** (MEMM, CRF, SVM): modella $P(Y | X)$ direttamente; generalmente più accurato in classificazione ma non genera dati.

19. Anatomia di un Parser

Con “**anatomia di un parser**” si intende la struttura modulare di un sistema di parsing, ovvero i componenti che lo compongono e le loro interazioni.

Componenti principali

1. **Grammatica / Modello:** specifica le strutture ammissibili (CFG, PCFG, dipendenze) e i loro pesi.
2. **Lessico / Dizionario:** associa le parole del vocabolario ai simboli grammaticali e alle loro proprietà.
3. **Algoritmo di parsing:** procedura che cerca la struttura ottimale (CKY, Earley, Viterbi, MST, Transition-based...).
4. **Modello di scoring:** calcola il punteggio di ogni struttura parziale o completa (locale in transition-based, globale in graph-based).
5. **Oracolo / Supervisione:** in training, fornisce il target corretto per apprendere i parametri.
6. **Decoder:** seleziona la struttura con punteggio massimo (argmax, beam search, ecc.).

Flusso di esecuzione

Input $\xrightarrow{\text{tokenizzazione}}$ Sequenza $\xrightarrow{\text{scoring}}$ Strutture candidate $\xrightarrow{\text{decoding}}$ Output

20. I 6 fenomeni notevoli del dialogo tra umani

1. Turn-Taking (gestione dei turni):

I parlanti si alternano in modo ordinato, spesso senza sovrapposizioni. Il cambio di turno è segnalato da cues prosodici, sintattici e gestuali. Modellato da Sacks, Schegloff e Jefferson (1974).

2. Speech Acts (atti linguistici):

Ogni enunciato non trasporta solo contenuto proposizionale ma compie un'azione: *asserzione, richiesta, promessa, ringraziamento*, ecc. (Austin, Searle). I dialogue systems devono riconoscere l'atto compiuto dall'utente.

3. Grounding (mutua comprensione):

I parlanti costruiscono attivamente un *Common Ground*: confermano, ripetono, riformulano per assicurarsi che il messaggio sia stato compreso. Segnali di grounding: "ok", "capisco", cenni del capo.

4. Repair (correzione):

I partecipanti al dialogo correggono errori propri (*self-repair*) o altrui (*other-repair*). Fondamentale per la robustezza: i dialogue systems devono gestire riformulazioni e correzioni.

5. Coerenza e struttura del discorso:

Il dialogo ha una struttura a livelli (scambi adiacenti, pairs come domanda-risposta, saluto-controsaluto). La *Discourse Representation Theory* (DRT) e la *Rhetorical Structure Theory* (RST) modellano tali strutture.

6. Implicature e massime conversazionali (Grice):

I parlanti comunicano più di quanto dicano letteralmente, seguendo le massime di

quantità, qualità, relazione e modo. Le *implicature conversazionali* sono inferenze pragmatiche cruciali per la comprensione (es. “Sai che ore sono?” = richiesta di dire l’ora).

21. Cosa è il PoS Tagging?

Il **Part-of-Speech (PoS) Tagging** è il task di assegnare a ciascun token di una frase la sua **categoria grammaticale** (o parte del discorso).

Classi principali

Le classi aperte includono **sostantivi** (N), **verbi** (V), **aggettivi** (ADJ), **avverbi** (ADV). Le classi chiuse includono **preposizioni** (P), **determinativi** (DET), **congiunzioni** (CONJ), **pronomi** (PRON), ecc.

Esempio (Universal POS tagset)

Il/DET gatto/NOUN nero/ADJ dorme/VERB sul/ADP tetto/NOUN ./ PUNCT

Sfide

- **Ambiguità**: “back” in inglese può essere NOUN, ADJ, VERB, ADV.
- **Parole sconosciute**: lemmi non visti in training (gestiti con feature morfologiche).

Approcci

- **Rule-based**: lessici + regole di disambiguazione.
- **Statistici**: HMM, MEMM, CRF.
- **Neurali**: BiLSTM + CRF, Transformer (BERT fine-tuned), stato dell’arte > 97% su inglese.

22. Cosa è il NER Tagging?

Il **Named Entity Recognition (NER)** è il task di identificare ed etichettare le *entità nominate* nel testo: nomi di persone, luoghi, organizzazioni, date, quantità, ecc.

Classi tipiche (CoNLL-2003)

Etichetta	Significato
PER	Persona
ORG	Organizzazione
LOC	Luogo geografico
MISC	Altro (es. nazionalità, prodotti)

Schema di annotazione (IOB2)

Ogni token riceve un tag nel formato B-TYPE (Begin), I-TYPE (Inside), O (Outside):

Mario/B-PER Rossi/I-PER lavora/O per/O Fiat/B-ORG a/O Torino/B-LOC ./O

Approcci

- **Rule-based:** gazetteers (liste di nomi) + pattern.
- **Statistici:** CRF con feature morfologiche e di contesto.
- **Neurali:** BiLSTM-CRF, BERT fine-tuned (stato dell'arte, F1 $\approx$ 93% su CoNLL-2003 EN).

Applicazioni

NER è un componente fondamentale in sistemi di Information Extraction, Question Answering, Knowledge Base Population e machine translation.